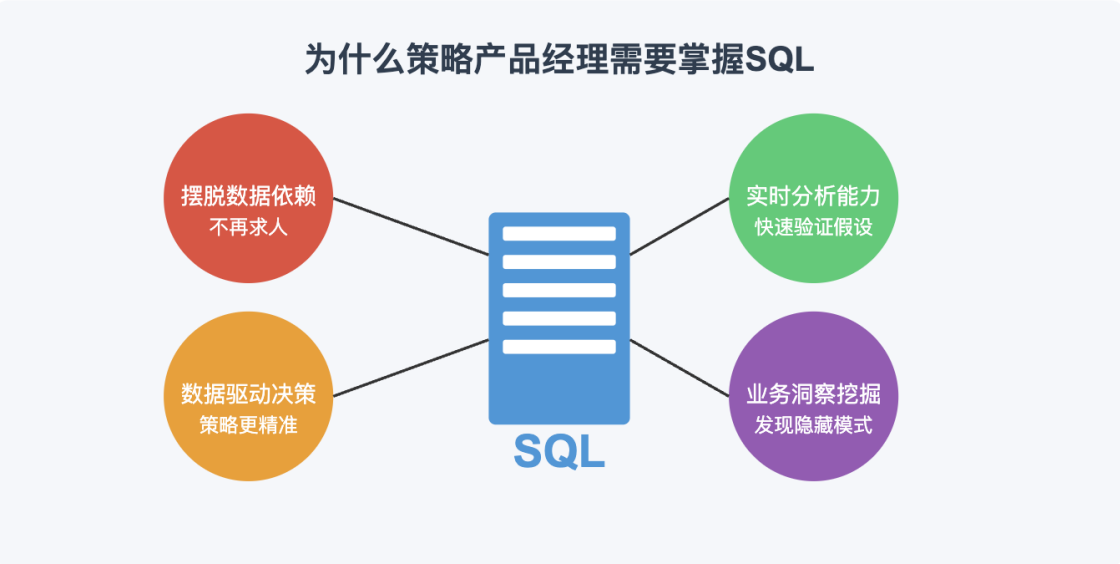


策略产品经理入门教程7：数据分析进阶之SQL基础与应用

各位好，我是  大厂策略产品搬砖狗，在大厂工作5年，一直从事搜索和推荐策略产品工作。这是我的策略产品经理入门教程第五篇，接下来我会分享更多策略产品的方法论和实战案例，25年底冲击万粉，感兴趣的话记得关注我，有什么问题也欢迎在评论区留言交流！

上一篇我们聊了怎么搭建基础指标体系，这篇想用一篇文章彻底帮你学会SQL，掌握数据分析的基础工具。作为策略产品经理，SQL是你必备的"硬核武器"！无论是分析用户行为、评估策略效果，还是挖掘业务洞察，SQL都是你与海量数据对话的基础语言。今天我们就来一起掌握这把利器，让数据为你的策略决策提供强大支持！

为什么策略产品经理必须掌握SQL？



策略产品经理不仅仅是业务与技术之间的桥梁，更是数据驱动决策的核心力量。以下四点解释了为什么SQL对我们如此重要：

-  **1. 摆脱数据依赖：**不必每次分析都去求助数据团队，提高工作效率，缩短策略响应时间
- 2. 实时分析能力：**随时验证假设、评估效果，让策略调整更加敏捷
- 3. 数据驱动决策：**基于事实而非猜测做决策，使策略更加精准有效
- 4. 业务洞察挖掘：**发现隐藏在数据背后的用户行为模式和业务机会

SQL入门：基本概念与语法

 SQL (Structured Query Language) 是一种用于管理关系型数据库的标准语言。对策略产品经理来说，我们主要关注其中的数据查询功能。

数据库基础概念

在了解SQL之前，我们需要掌握几个基本概念：

- **数据库(Database)**：存储结构化数据的集合
- **表(Table)**：具有行和列的数据集合，如上图中的"user_behavior"表
- **列/字段(Column)**：表中的一个属性，如user_id、event_type等
- **行/记录(Row)**：表中的一条完整数据记录
- **主键(Primary Key)**：唯一标识表中每条记录的字段，如上例中可能是user_id+timestamp的组合
- **外键(Foreign Key)**：用于关联其他表的字段

SQL基础语法：SELECT语句

作为策略产品经理，最常用的SQL语句就是SELECT查询。它的基本语法如下：

sql示例

```
1  SELECT 字段1, 字段2, ...
2  FROM 表名
3  WHERE 条件
4  GROUP BY 分组字段
5  HAVING 分组后筛选条件
6  ORDER BY 排序字段 [ASC|DESC]
7  LIMIT 限制返回记录数;
```

让我们一步步拆解这个语法：

1. 基本查询：SELECT & FROM

代码块

```
1  -- 查询所有字段
2  SELECT * FROM user_behavior;
3
4  -- 查询特定字段
```

```
5 SELECT user_id, event_type, item_id FROM user_behavior;
```

2. 条件筛选：WHERE

代码块

```
1  -- 查询特定用户的行为
2  SELECT * FROM user_behavior WHERE user_id = 12345;
3
4  -- 查询特定时间段的行为
5  SELECT * FROM user_behavior WHERE timestamp >= '2023-09-01' AND timestamp <
   '2023-09-02';
6
7  -- 查询特定类型的行为
8  SELECT * FROM user_behavior WHERE event_type IN ('click', 'like');
```

3. 聚合与分组：GROUP BY & 聚合函数

SQL聚合函数与分组示例

原始数据: user_behavior表				
user_id	event_type	item_id	timestamp	platform
101	click	A01	2023-09-01	android
102	view	B02	2023-09-01	ios
101	like	A01	2023-09-01	android
103	click	C03	2023-09-02	web

```
SELECT event_type, COUNT(*) as event_count FROM user_behavior GROUP BY event_type
```

聚合后的结果	
event_type	event_count
click	2
view	1
like	1

聚合函数和GROUP BY是策略分析的核心。通过它们，我们可以汇总数据，发现模式。

常用的聚合函数：

- **COUNT()**: 计数
- **SUM()**: 求和
- **AVG()**: 平均值

- **MAX()/MIN()**: 最大/最小值
- **DISTINCT**: 去重

示例:

代码块

```
1  -- 统计各类行为的发生次数
2  SELECT event_type, COUNT(*) AS event_count
3  FROM user_behavior
4  GROUP BY event_type;
5
6  -- 统计不同平台的用户数量
7  SELECT platform, COUNT(DISTINCT user_id) AS user_count
8  FROM user_behavior
9  GROUP BY platform;
10
11 -- 计算每个用户的平均行为次数
12 SELECT user_id, COUNT(*) AS behavior_count
13 FROM user_behavior
14 GROUP BY user_id;
```

4. 排序与限制: ORDER BY & LIMIT

代码块

```
1  -- 查找最活跃的前10名用户
2  SELECT user_id, COUNT(*) AS activity_count
3  FROM user_behavior
4  GROUP BY user_id
5  ORDER BY activity_count DESC
6  LIMIT 10;
7
8  -- 按时间排序查看最新行为
9  SELECT * FROM user_behavior
10 ORDER BY timestamp DESC
11 LIMIT 100;
```

5. 复杂条件: HAVING

HAVING与WHERE类似, 但用于对分组后的结果进行筛选。

代码块

```
1  -- 找出行为次数超过10次的用户
2  SELECT user_id, COUNT(*) AS behavior_count
3  FROM user_behavior
4  GROUP BY user_id
5  HAVING behavior_count > 10;
```

策略产品经理必会的SQL进阶技巧

掌握基础后，接下来是一些策略分析中常用的进阶技巧：

1. 表连接：JOIN操作

SQL表连接(JOIN)操作

user_behavior表		
user_id	event_type	item_id
101	click	A01
102	view	B02
103	like	C03

user_profile表		
user_id	age	gender
101	25	M
102	30	F
104	22	M

INNER JOIN: 两表共有的记录

```
SELECT b.user_id, b.event_type, p.age, p.gender FROM user_behavior b INNER JOIN user_profile p ON b.user_id = p.user_id
```

连接后的结果			
user_id	event_type	age	gender
101	click	25	M
102	view	30	F

注：除了INNER JOIN，还有LEFT JOIN(保留左表所有行)、RIGHT JOIN(保留右表所有行)、FULL JOIN(保留所有行)

JOIN操作是将多张表连接到一起的方法，对于分析用户画像、内容偏好等复杂问题非常重要。

常见的JOIN类型：

- **INNER JOIN**：仅返回两表中匹配的行
- **LEFT JOIN**：返回左表所有行，右表匹配的行
- **RIGHT JOIN**：返回右表所有行，左表匹配的行
- **FULL JOIN**：返回两表的所有行

示例：将用户行为与用户画像信息连接

代码块

```
1  -- 分析不同性别用户的行为差异
```

```

2  SELECT p.gender, b.event_type, COUNT(*) AS behavior_count
3  FROM user_behavior b
4  INNER JOIN user_profile p ON b.user_id = p.user_id
5  GROUP BY p.gender, b.event_type
6  ORDER BY p.gender, behavior_count DESC;
7
8  -- 查找高活跃但未付费的用户(潜在转化目标)
9  SELECT b.user_id, p.age, p.gender, COUNT(*) AS activity_count
10 FROM user_behavior b
11 LEFT JOIN payment_record pay ON b.user_id = pay.user_id
12 INNER JOIN user_profile p ON b.user_id = p.user_id
13 WHERE pay.payment_id IS NULL
14 GROUP BY b.user_id, p.age, p.gender
15 HAVING activity_count > 10
16 ORDER BY activity_count DESC;

```

2. 子查询与嵌套查询

子查询是查询中的查询，可以帮助我们处理更复杂的数据分析需求。

代码块

```

1  -- 查找活跃度高于平均值的用户
2  SELECT user_id, COUNT(*) AS activity_count
3  FROM user_behavior
4  GROUP BY user_id
5  HAVING COUNT(*) > (
6      SELECT AVG(activity) FROM (
7          SELECT COUNT(*) AS activity
8          FROM user_behavior
9          GROUP BY user_id
10     ) AS avg_activity
11 );
12
13 -- 查找近期没有活动的高价值用户
14 SELECT user_id, total_payment
15 FROM (
16     SELECT user_id, SUM(amount) AS total_payment
17     FROM payment_record
18     GROUP BY user_id
19     HAVING total_payment > 1000
20 ) AS high_value_users
21 WHERE user_id NOT IN (
22     SELECT DISTINCT user_id
23     FROM user_behavior

```

```
24 WHERE timestamp >= '2023-09-01'
25 );
```

3. 窗口函数：分析趋势与排名

窗口函数是策略分析的强大工具，可以在不改变结果集行数的情况下进行计算，非常适合趋势分析和排名。

SQL窗口函数应用示例

原始数据: item_sales表			
item_id	category	sales_date	amount
A01	Electronics	2023-08-01	1500
B02	Clothing	2023-08-01	800
A01	Electronics	2023-08-02	1800
C03	Home	2023-08-02	1200

```
SELECT *, RANK() OVER (PARTITION BY sales_date ORDER BY amount DESC) AS daily_rank
```

使用窗口函数后的结果				
item_id	category	sales_date	amount	daily_rank
A01	Electronics	2023-08-01	1500	1
B02	Clothing	2023-08-01	800	2

4. 常用日期函数与时间分析

策略产品常常需要分析时间序列数据，SQL提供了强大的日期处理函数。

```
代码块
1  -- 按周分析用户活跃度
2  SELECT
3      EXTRACT(YEAR FROM timestamp) AS year,
4      EXTRACT(WEEK FROM timestamp) AS week,
5      COUNT(DISTINCT user_id) AS weekly_active_users
6  FROM user_behavior
7  GROUP BY year, week
8  ORDER BY year, week;
9
10 -- 用户留存分析
11 SELECT
12     first_date,
```

```

13     COUNT(DISTINCT user_id) AS new_users,
14     COUNT(DISTINCT CASE WHEN days_diff = 1 THEN user_id END) AS day1_retained,
15     COUNT(DISTINCT CASE WHEN days_diff = 7 THEN user_id END) AS day7_retained,
16     COUNT(DISTINCT CASE WHEN days_diff = 30 THEN user_id END) AS day30_retained
17 FROM (
18     SELECT
19         a.user_id,
20         MIN(DATE(a.timestamp)) AS first_date,
21         DATEDIFF(DATE(b.timestamp), MIN(DATE(a.timestamp))) AS days_diff
22     FROM user_behavior a
23     JOIN user_behavior b ON a.user_id = b.user_id
24     GROUP BY a.user_id, DATE(b.timestamp)
25 ) retention
26 GROUP BY first_date
27 ORDER BY first_date;

```

5. 分组集合：GROUP BY GROUPING SETS, ROLLUP, CUBE

当我们需要在一个查询中同时进行多维度的分组统计时，分组集合非常有用。

代码块

```

1  -- 使用GROUPING SETS同时获取不同维度的统计
2  SELECT
3      COALESCE(platform, 'All Platforms') AS platform,
4      COALESCE(event_type, 'All Events') AS event_type,
5      COUNT(*) AS event_count
6  FROM user_behavior
7  GROUP BY GROUPING SETS (
8      (platform, event_type),
9      (platform),
10     (event_type),
11     ()
12 )
13 ORDER BY platform, event_type;
14
15 -- 使用ROLLUP进行层级汇总
16 SELECT
17     date_trunc('month', timestamp) AS month,
18     platform,
19     event_type,
20     COUNT(*) AS event_count
21 FROM user_behavior
22 GROUP BY ROLLUP (
23     date_trunc('month', timestamp),

```



```
24     platform,
25     event_type
26 )
27 ORDER BY month, platform, event_type;
```

6. 正则表达式与文本分析

在策略产品工作中，常常需要分析用户搜索词、内容标题等文本信息，此时正则表达式是强大的工具。

代码块

```
1  -- 查找包含特定关键词的搜索记录
2  SELECT
3      search_query,
4      COUNT(*) AS search_count
5  FROM search_logs
6  WHERE search_query REGEXP '手机|电脑|数码'
7  GROUP BY search_query
8  ORDER BY search_count DESC;
9
10 -- 提取搜索词中的品牌名称
11 SELECT
12     REGEXP_EXTRACT(search_query, '(苹果|华为|小米|OPPO|vivo)') AS brand,
13     COUNT(*) AS brand_search_count
14 FROM search_logs
15 WHERE REGEXP_EXTRACT(search_query, '(苹果|华为|小米|OPPO|vivo)') IS NOT NULL
16 GROUP BY brand
17 ORDER BY brand_search_count DESC;
```

策略产品场景中的SQL实战案例

让我们通过几个真实的策略产品场景，来看看如何运用SQL解决实际问题：

案例1：搜索策略优化 - 长尾搜索词挖掘

在搜索产品中，长尾搜索词往往代表了用户的特定需求，但可能因为搜索结果不佳导致转化率低。下面的SQL可以帮助挖掘这些潜在的优化机会：

代码块

```
1  -- 识别高频但低转化的长尾搜索词
```

```

2  SELECT
3      search_query,
4      COUNT(*) AS search_count,
5      COUNT(DISTINCT user_id) AS user_count,
6      SUM(CASE WHEN event_type = 'click' THEN 1 ELSE 0 END) AS click_count,
7      ROUND(SUM(CASE WHEN event_type = 'click' THEN 1 ELSE 0 END) * 100.0 /
8      COUNT(*), 2) AS click_rate,
9      ROUND(AVG(result_count), 0) AS avg_result_count
10 FROM search_events
11 WHERE
12     timestamp >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
13     AND LENGTH(search_query) > 10 -- 长查询词
14 GROUP BY search_query
15 HAVING
16     search_count >= 100 -- 高频
17     AND click_rate < 30 -- 低点击率
18     AND avg_result_count > 0 -- 有结果但质量可能不高
19 ORDER BY search_count DESC, click_rate ASC
20 LIMIT 100;

```

案例2：推荐策略效果评估 - A/B测试结果分析

策略产品经常需要通过A/B测试来评估新策略的效果。以下SQL可以帮助我们进行详细的测试结果分析：

代码块

```

1  -- A/B测试效果分析
2  WITH user_metrics AS (
3      SELECT
4          u.user_id,
5          u.experiment_group, -- 'A' 或 'B'
6          COUNT(DISTINCT CASE WHEN b.event_type = 'view' THEN b.session_id END)
7      AS session_count,
8          COUNT(DISTINCT CASE WHEN b.event_type = 'click' THEN b.item_id END) AS
9      clicked_items,
10         COUNT(DISTINCT CASE WHEN b.event_type = 'purchase' THEN b.order_id
11     END) AS purchase_count,
12         SUM(CASE WHEN b.event_type = 'purchase' THEN b.amount ELSE 0 END) AS
13     total_spend
14     FROM user_experiment u
15     LEFT JOIN user_behavior b ON u.user_id = b.user_id
16     WHERE
17         u.experiment_id = 'rec_algo_v2_test'
18         AND b.timestamp BETWEEN u.start_time AND u.end_time

```

```

15     GROUP BY u.user_id, u.experiment_group
16 )
17 SELECT
18     experiment_group,
19     COUNT(*) AS user_count,
20     ROUND(AVG(session_count), 2) AS avg_sessions,
21     ROUND(AVG(clicked_items), 2) AS avg_clicks,
22     ROUND(AVG(purchase_count), 4) AS avg_purchases,
23     ROUND(SUM(purchase_count) * 100.0 / COUNT(*), 2) AS conversion_rate,
24     ROUND(SUM(total_spend) / NULLIF(SUM(purchase_count), 0), 2) AS
    avg_order_value,
25     ROUND(SUM(total_spend) / COUNT(*), 2) AS arpu
26 FROM user_metrics
27 GROUP BY experiment_group
28 ORDER BY experiment_group;

```

SQL效率优化技巧

在大数据环境中，SQL查询效率直接影响策略分析和调整的速度。以下是一些提高查询效率的技巧：

1. 只查询需要的数据

代码块

```

1  -- 避免使用SELECT *, 只查询需要的字段
2  -- 低效写法
3  SELECT * FROM user_behavior WHERE event_type = 'click';
4
5  -- 高效写法
6  SELECT user_id, item_id, timestamp FROM user_behavior WHERE event_type =
    'click';

```

2. 善用索引

确保WHERE、JOIN、ORDER BY子句中使用的字段都有适当的索引。

代码块

```

1  -- 在经常查询的字段上创建索引
2  CREATE INDEX idx_user_behavior_user_id ON user_behavior(user_id);
3  CREATE INDEX idx_user_behavior_event_time ON user_behavior(event_type,
    timestamp);

```

3. 分区表和分桶表

对于超大表，使用分区可以大幅提高查询效率。

代码块

```
1  -- 创建按日期分区的表
2  CREATE TABLE user_behavior_partitioned (
3      user_id BIGINT,
4      event_type STRING,
5      item_id BIGINT,
6      timestamp TIMESTAMP
7  )
8  PARTITIONED BY (dt STRING)
```

4. 预计算与物化视图

对于复杂的聚合计算，可以使用预计算和物化视图提高响应速度。

sql

代码块

```
1  -- 创建每日用户活跃度物化视图
2  CREATE MATERIALIZED VIEW daily_active_users AS
3  SELECT
4      DATE(timestamp) AS day,
5      COUNT(DISTINCT user_id) AS dau
6  FROM user_behavior
7  GROUP BY DATE(timestamp);
```

5. 使用EXPLAIN分析执行计划

在执行复杂查询前，使用EXPLAIN来分析SQL的执行计划，找出潜在的性能瓶颈。

代码块

```
1  -- 分析查询执行计划
2  EXPLAIN
```

```
3  SELECT
4      u.gender,
5      COUNT(DISTINCT b.user_id) AS user_count
6  FROM user_behavior b
7  JOIN user_profile u ON b.user_id = u.user_id
8  WHERE b.timestamp >= '2023-09-01'
9  GROUP BY u.gender;
```

策略产品经理的SQL进阶路径

作为策略产品经理，掌握SQL是一个持续提升的过程。以下是你的SQL技能进阶路径：



- 1. 基础阶段：**掌握SELECT查询基本语法，能够完成简单的数据统计
- 2. 进阶阶段：**熟练使用JOIN、子查询、窗口函数等解决复杂业务问题
- 3. 专家阶段：**能够优化SQL性能，设计数据模型，构建自动化报表系统
- 4. 领导阶段：**能够建立数据驱动的决策文化，指导团队进行高效数据分析

记住，SQL只是工具，真正的价值在于你能从数据中发现什么洞察，以及如何将这些洞察转化为产品策略的优化。祝你在策略产品的道路上越走越远！

下一篇预告：策略产品经理入门教程8 - 用户画像构建与应用营

在下一篇教程中，我们将探讨如何基于用户行为和价值进行科学的用户画像构建，并制定差异化的策略，实现精细化运营，敬请期待！